Factorial Recursive –

Space Complexity(Detailed Analysis)

Program:

```
#include <iostream>
using namespace std;

int factorial(int n)
{
    if (n == 0 || n == 1)
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}

int main()
{
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial of " << n << " = " << factorial(n);
    return 0;
}
```

Space Complexity

1. *Input Space Complexity*: $\boldsymbol{O}(\mathbf{1})$

Input Space Complexity refers to the memory strictly required to store the initial data or variables provided to the algorithm.

- ***Parametric Input Space*** $(\boldsymbol{O}(\mathbf{1}))$: *This is the memory required specifically for the parameters passed into function signature. In function* int factorial(int n), *the only parameter is a single integer* n. *Regardless of whether calculating the factorial of* 5 *or* 500, *a standard integer always takes up a fixed amount of memory (typically* 4 *bytes in* $C++$). *Therefore, the parametric space is constant, or* $\boldsymbol{O}(\mathbf{1})$.

- ***Holistic Input Space*** $(\boldsymbol{O}(\mathbf{1}))$: *This refers to the overall, global memory footprint of the input data entire program needs to function, regardless of how it's passed around. In* main() *declaration* int n; *to hold the user's input. The totality of the data being processed is just this one integer. Because the size of the input data structure doesn't grow (it's always one number, not an array of n numbers), the holistic input space is also* $\boldsymbol{O}(\mathbf{1})$.

2. $Auxiliary\ Space\ Complexity: O(n)$

$Auxiliary\ space\ =\ extra\ memory\ used\ during\ execution$
$(excluding\ input\ storage)$

$Each\ stack\ frame\ stores:$

- $parameter\ n$
- $return\ address$
- $local\ variables\ (none\ here, but\ still\ stack\ overhead)$

$$Each\ frame\ =\ \Theta(1) space\ per\ recursive\ call$$

$factorial(n)\ =\ n\ *\ factorial(n-1)$

$Factorial(n) \rightarrow This\ is\ a$ *linear chain*, $not\ a\ branching\ tree.$

<u>*Conceptual clearance first*</u>

1. $Edges\ are\ the\ path\ between\ two\ nodes\ ,but\ after$

$execution\ of\ base\ returning\ `1`\ \left(output\ value\ of\ leaf\ node\ \begin{bmatrix} Base \\ Case \end{bmatrix}\right)$

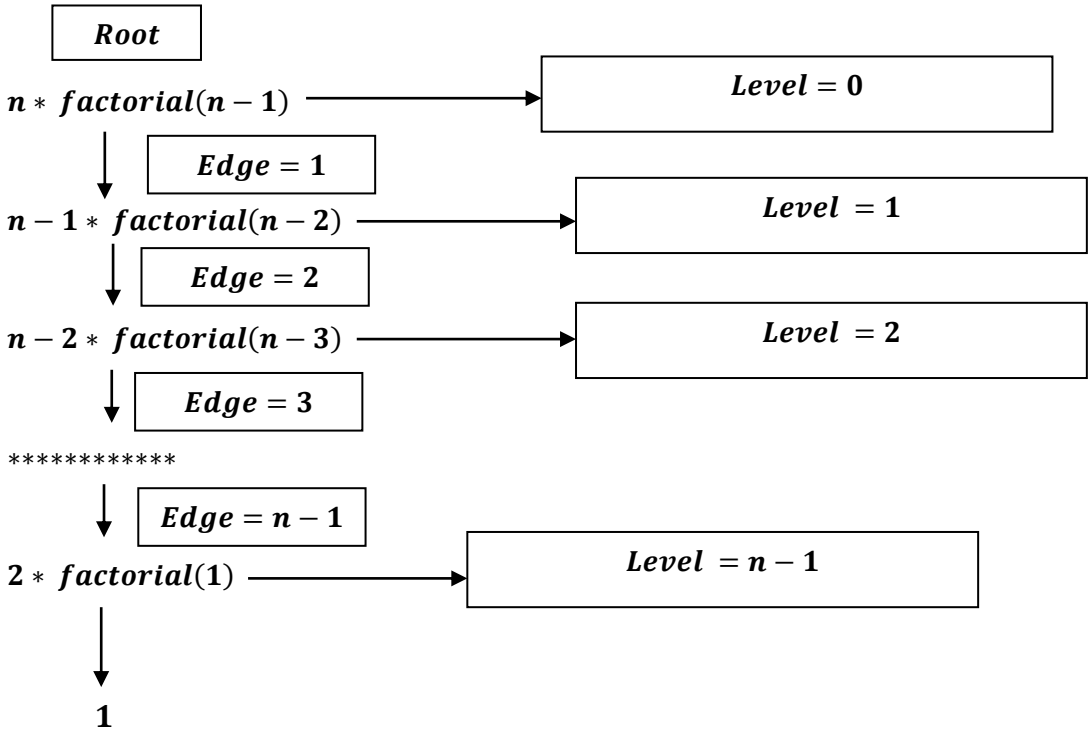
$is\ not\ a\ node\ but\ nodes\ are\ recursive\ call\ stack\ frames$

$from\ root\ to\ leaf\ node[Here\ , Fact(0)\ and\ Fact(1)]$.

2. $Levels\ are\ group\ of\ nodes\ at\ the\ same\ distance\ from\ the\ root.$

3. $Depth\ or\ Maximum\ Depth\ is\ the\ maximum\ edges\ from\ root$

$node\ to\ leaf\ node.$



For Depth Calculation

$\boldsymbol{Basically\ , it\ ends\ with\ base\ case\ Fact(1).}$

$\boldsymbol{n - k = 1[user\ input\ `n` - Edges(k) = 1]}$

$\boldsymbol{\Longrightarrow k = n - 1}$

$\boldsymbol{As\ it\ is\ not\ a\ tree\ , it\ is\ a\ linear\ chain,}$

$\boldsymbol{Number\ of\ \ edges\ in\ Fact(n) = k = n - 1.}$

$\boldsymbol{N(Nodes) = Edges(k) + 1(Root)}$

$\boldsymbol{\Longrightarrow N = (n-1) + 1 = n\ \ , then:}$

$$Hence\ , \qquad the\ Maximum\ Depth = No.of\ Edges = n - 1.$$

$Based\ on\ Active\ stackframe$:

$$\therefore Total\ Auxiliary\ Space$$
$$= (\ Maximum\ Depth + 1) \times \begin{pmatrix} Space\ per \\ Recursive\ Call \end{pmatrix}$$

$$\begin{bmatrix} Here + 1\ for\ Root\ Node\ , which\ is\ also\ an\ active \\ stack\ frame \end{bmatrix}$$

$$= \Theta(n - 1 + 1) \times \Theta(1)$$
$$= \Theta(n) \times \Theta(1)$$

$$= \Theta(n)$$

$More\ clearly, fact(n) \rightarrow fact(n-1) \rightarrow \ldots \rightarrow fact(1)$

$Maximum\ Depth = Number\ of\ active\ calls\ at\ deepest\ point = n(Also\ correct).$

$if\ avoided\ Edge\ and\ Level\ counting\ technique\ .As\ it$

$is\ not\ a\ tree, but\ edge\ and\ level\ coounting\ is\ very\ important$

$in\ trees.$

$\boldsymbol{Total\ Space\ Complexity}$:

$$\boldsymbol{Input\ Space + Auxiliary\ Space = \Theta(1) + \Theta(n)}$$
$$\boldsymbol{= \Theta(n)}.$$